Technical debt review - 2.7.x

Technical debt review and recommendations

Date: 12 November 2026

Reviewer role: Programme management / technical debt

Release under review: `2.7.0` (development `172.16.16.44`)

Method: file-size and hotspot inventory, test/CI review, incident review of the 12 November availability stall, security backlog, dependency review, and the data-quality audit.

Executive summary

The platform is feature-rich and the delivery discipline is above average: supply-chain locking, lab validation, CSP hashing, documentation builds, and a large backend test suite are already in place. The debt is concentrated, not spread: four very large files hold most of the complexity, the frontend has no unit or E2E gate in CI, lint is non-blocking, and the single-worker backend has no guardrail against blocking work on the event loop.

The 12 November outage was not a feature bug. A lab spawn/teardown held a database session while the instructor dashboard fired a burst of analytics requests. The 4+4 pool exhausted, the next auth lookup blocked the event loop for 30 seconds, and every page appeared to hang. The immediate fixes are in place; the architectural pattern that allowed it is the top debt item.

Debt inventory

Area	Evidence	Risk	Effort	Priority
Monolithic admin UI	`Admin.vue` 14,389 lines; `InstructorPanel.vue` 4,626; `TrackDetail.vue` 4,432	Review cost, merge conflicts, regression risk	High	P1
Monolithic backend routers	`admin.py` 4,496 lines; `docker_manager.py` 4,048; `courses.py` 2,657; `analytics.py` 2,382	Hard to test in isolation; slow review	High	P1
Event-loop discipline	Sync DB/Docker work in async endpoints; pool exhaustion incident	Total platform stall	Medium	P1
Frontend test gap	No unit tests, no Playwright job in the default CI; `ui-smoke.yml` needs credentials	Button/flow regressions reach users	Medium	P1

Area	Evidence	Risk	Effort	Priority
Lint gate	51 ruff findings; CI `continue-on-error: true`; 132 ESLint warnings	Drift and dead code accumulate	Low	P2
N+1 queries	91 relationships, few eager loads; analytics/dashboard routers	Latency at scale	Medium	P2
Retention	Ledger/activity/flag_attempts grow without a prune job	Storage and review-queue noise	Low	P2
Dependency currency	Postgres 15 runtime vs 17 CI; Python 3.11; base images tag-pinned	Upgrade crunch, supply-chain drift	Medium	P2
Auth storage	JWT in localStorage (mitigated by CSP + DOMPurify)	XSS token theft if CSP regresses	High	P2
Deferred hardening	RangeBox pinning/supervisord; Valkey `requirepass`; fail-closed revocation; username timing	Security depth	Medium	P2
Content scale	169 labs, 160 workbook pages, 43 chapters; many `review_status` pending live walkthrough	Content quality drift	Medium	P3
Repo sprawl	`opencyberrange`, `-ha`, `-next`, `-v1.1-fix`, `backups` on the workstation	Wrong-tree edits, stale docs	Low	P3

Top recommendations

1. Split `Admin.vue` and `admin.py` by domain (P1)

Extract the System, Users, Courses, Events, Achievements, and Question Bank panels into components with their own props/events. On the backend, split `admin.py` into `admin_users.py`, `admin_system.py`, `admin_content.py`, and `admin_reports.py` behind the same router prefix. Acceptance: no file above 1,500 lines; routes unchanged; tests pass.

2. Make the event loop non-blocking by rule (P1)

Adopt one rule: no synchronous DB, Docker, filesystem, or report generation in an `async def` endpoint. Move work to `asyncio.to_thread` / `run_in_executor`, or make the endpoint `def` so FastAPI uses the threadpool. Add a CI check that flags `SessionLocal()` or `docker_manager.` calls inside `async def` bodies. Acceptance: the pool-burst test and the scheduler tick stay under 50 ms p95.

3. Add a real frontend gate (P1)

Wire a Playwright job into CI with a seeded test database: student, instructor, and admin login; dashboard, exercises, profile, admin System, question banks, achievements, events, and the CyberChef panel. Keep `ui-smoke.mjs` and extend

it rather than creating a second harness. Acceptance: the job is required and runs in under 10 minutes.

4. Turn lint into a ratchet (P2)

Fix the 51 ruff findings (36 auto-fixable) and remove `continue-on-error` from the ruff step. For ESLint, fail on errors now and track warnings with a budget that must not increase. Acceptance: zero ruff errors; warning count frozen.

5. Eager-load the hot routers (P2)

Audit `analytics.py`, `dashboard.py`, `admin.py`, and `courses.py` for loops over relationships. Use `selectinload` for collections and `joinedload` for many-to-one; use `count()` where rows are not needed. Gate with the existing stress test. Acceptance: p95 latency and query count improve on the roster and dashboard scenarios.

6. Schedule retention (P2)

Add a scheduler job for `data_integrity_ledger`, `activity_events`, and `flag_attempts`, with a documented retention window and a run record in the ledger. Acceptance: row counts stabilize; the data analyst review queue is smaller.

7. Separate report signing and rotate secrets (P2)

Set `REPORT_SIGNING_KEY` independently of `JWT_SECRET`; document a rotation runbook for `WG_ENCRYPTION_KEY` and `REPORT_SIGNING_KEY`. Acceptance: a rotation drill completes without losing access to stored secrets.

8. Finish the deferred security depth (P2)

RangeBox supervisord + digest pinning, Valkey `requirepass`, fail-closed revocation for privileged roles, username-enumeration timing, and digest pinning for base images. Acceptance: each item has a test or a recorded accepted risk.

9. Upgrade with a rehearsal (P2)

Postgres 15 -> 17 and Python 3.11 -> 3.12, one at a time, on a restored copy. Record timings and rollback steps. Acceptance: a staging restore and upgrade complete inside the maintenance window.

10. Keep the release gate honest (P1 process)

Use `docs/PROD-DEPLOYMENT-VALIDATION-MATRIX.md` for every promotion. A release

does not ship with NOT RUN rows. Acceptance: 100% required rows PASS and signed.

Guardrails to stop regrowth

1. One PR per numbered item; no drive-by features.
2. A stress-test baseline diff for anything on a request path.
3. No new inline migrations in `main.py`; Alembic only.
4. Every new endpoint has a role test and a rate-limit decision.
5. Every new UI surface has a loading, empty, error, and permission-denied state.
6. The production matrix is the only release gate; chat approvals do not count.

What not to do

- Do not rewrite the frontend or adopt a new framework to solve file size.
- Do not add a second theme system or a second test harness.
- Do not raise the DB pool again without fixing long-held sessions.
- Do not promote to production while restore, upgrade, and rollback drills are

NOT RUN.